

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/318328095>

Recommending Programming Languages by Identifying Skill Gaps Using Analysis of Experts. A Study of Stack Overflow

Conference Paper · July 2017

DOI: 10.1145/3099023.3099040

CITATIONS

13

READS

4,230

5 authors, including:



Ifeoma Adaji

University of British Columbia - Okanagan

67 PUBLICATIONS 691 CITATIONS

SEE PROFILE



Julita Vassileva

University of Saskatchewan

381 PUBLICATIONS 11,133 CITATIONS

SEE PROFILE



Ralph Deters

University of Saskatchewan

239 PUBLICATIONS 5,244 CITATIONS

SEE PROFILE

Recommending Programming Languages By Identifying Skill Gaps Using Analysis of Experts. A Study Of Stack Overflow

Obaro Odiete
University of Saskatchewan
Saskatoon, Saskatchewan
obaro.odiete@usask.ca

Tanvi Jain
University of Saskatchewan
Saskatoon, Saskatchewan
tanvi.jain@usask.ca

Ifeoma Adaji
University of Saskatchewan
Saskatoon, Saskatchewan
ifeoma.adaji@usask.ca

Julita Vassileva
University of Saskatchewan
Saskatoon, Saskatchewan
jiv@cs.usask.ca

Ralph Deters
University of Saskatchewan
Saskatoon, Saskatchewan
deters@cs.usask.ca

ABSTRACT

The increasing variety of programming languages available to computer programmers has led to the discussion of what language(s) should be learned. A key point in the choice of a programming language is the availability of support from experienced programmers. In this paper, we explore the use of graph theory in recommending programming languages to novice and expert programmers in a question and answer collaborative learning environment, Stack Overflow. Using social network analysis techniques, we investigate the relationship between experts (using an expertise graph) in different programming languages in order to identify what languages can be recommended to novice and experienced programmers. In addition, we explore the use of the expertise graph in inferring the importance of a programming language to the community. Our results suggest that programming languages can be recommended within organizational borders and programming domains. In addition, a high number of experts in a programming language does not always mean that the language is popular. Furthermore, disconnected nodes in the expertise graph suggest that experts in some programming languages are primarily on Stack Overflow to support that language only and do not contribute to questions or answers in other languages. Finally, developers are comfortable with mastering a single, general purpose language. The results of our study can help educators and stake holders in computer education to understand what programming languages can be suggested to students and what languages can be taught and learned together.

KEYWORDS

Learning, Recommendation, Stack Overflow

1 INTRODUCTION

There are several hundreds of programming languages in existence today¹ and more are still being created. This increasing variety of programming languages available to developers leads to several discussions² on the choice of language(s) to learn. This question on the choice of languages has also been brought up in the literature; In [13], Morrison et al. suggested that for personal development, questions such as “What [programming language] do I need to

learn?” and “Who can help me learn it?” should be asked. In this paper, we aim to answer the first question for both novice and experienced programmers. We define novice programmers as individuals without any programming experience while experienced programmers are those with some programming experience, in one or more languages. Novice programmers are faced with the question of what languages they should start with while experienced programmers wonder which language to learn next. There are different factors that can be considered in selecting the programming languages to learn such as popularity, efficiency, ease of learning, and availability of experts. In this paper, we make use of data extracted from Stack Overflow³, a popular question and answer (Q&A) collaborative learning environment, to identify gaps in programming skills and recommend the language a novice or experienced programmer could learn.

In this paper, we aim to answer the following research questions:

- RQ1: What new programming language can be recommended to an experienced programmer based on the expertise graph?
- RQ2: Can we infer the importance of a programming language from the expertise graph?
- RQ3: What can be inferred from isolated nodes and loop-back edges of the expertise graph?

Using social network analysis techniques, our results suggest that Programming languages can be recommended within organizational borders and programming domain. In addition, a high number of experts in a programming language does not always mean the language is popular. Furthermore, disconnected nodes in the expertise graph include not only older programming languages but also new ones whose experts are primarily on Stack Overflow to support that language. Finally, programmers are comfortable with mastering a single general purpose language.

The main contributions of this paper include defining an expertise graph for Stack Overflow, and recommending programming skills to fill a user’s knowledge gap based on the expertise graph.

¹<https://www.quora.com/How-many-programming-languages-are-there-in-the-world>

²<http://blog.udacity.com/2015/05/pick-your-first-programming-language.html>

³<http://stackoverflow.com>

2 RELATED WORK

2.1 Stack Overflow

Stack Overflow is a Q&A collaborative learning environment where users can ask and answer specific computer programming questions. When users post questions, they can select up to five keywords broadly identifying the domain of their question. These keywords, known as tags, could be specific programming languages (Java, C++, Erlang), programming frameworks (AngularJS, Django) or even general topics (databases, authentication). score⁴. The quality of contributions is kept high by rewarding high quality questions using reputation scores, badges and privileges. Although only the asker can mark a question as answered, others can vote the question and answers up or down. Voting is regulated by requiring higher privilege and costing some reputation Research[12] have shown that this motivational strategy helps keep the network active and of high quality

The strong connection between Stack Overflow and programmers cannot be overemphasized. According to Treude et al. [18], Stack Overflow is a very important resource for developers which provides alternatives to official product documentations when they are limited or unavailable. Squire [17] studied software projects moving to Stack Overflow in order to understand why and if the move was successful. Bacchelli et al. [1] introduced a system for recommending Stack Overflow knowledge to developers right in the Integrated Development Environment (IDE). They [1] argued that their system, Seahawk, would allow developers to access answers on Stack Overflow without leaving the IDE. In [13], the authors used samples of Stack Overflow dataset and showed a positive correlation between user's age and reputation. They pointed out that within the boundaries of their studies, it appears older programmers can still learn new technologies. Similarly, our work aims at understanding developer interaction within the context of Stack Overflow and using that to improve their off-line experience.

2.2 Stack Overflow Tags

There have been several works using Stack Overflow tags [2, 14, 16]. In particular, Bosu et al. [5] proposed strategies for contributors in the Stack Overflow community to build reputation scores. Using the community detection algorithm in [4], they grouped the Stack Overflow tags into 14 categories and reported result on these tags and categories using different metrics such as the number of questions, experts and speed of response. Their aim was to identify tag categories and strategies for contributors in Stack Overflow to quickly build reputation score. Similarly, in our work we would categorize programming language specific tags to provide programming skills recommendation to programmers. Another related work is by MacLeod [11] who studied the relationship between variety of tags a user contributes to in form of questions, answers or comments and the user's reputation score. Their findings on data from Stack Exchange network suggest a positive correlation. Barua et al. [2] used Latent Dirichlet Allocation (LDA) to automatically detect the major topics in Stack Overflow. They argued that the questions tags have several drawbacks that would not help in their research goals. Nevertheless, in our work, we use tags to limit the programming

languages concentrated on as we believe that despite their short comings, the questions tags would be sufficient for our research.

2.3 Graph Analysis and Visualization

Many tools exist for analyzing and visualizing networks, however we used Gephi [3] for visualizing our network graph because of its flexibility and NetworkX [9] for analyzing the graph properties, because most of the algorithms it implements supports graphs with weighted edges.

Gephi is an open source software for exploring and manipulating graphs. It has been used repeatedly in Stack Overflow [5, 11] and other Q&A networks [8]. NetworkX [9] is a free python software package used for studying graph. It contains implementation of many graph algorithms for computing properties of graphs such as centrality, clustering and so on.

3 RESEARCH DESIGN AND METHODOLOGY

In this section, we describe how the research was designed and carried out. In particular, we describe the dataset that was used and the programming expertise graph which forms the foundation of our exploration.

3.1 Dataset Description

This study was carried out on 47 programming languages published online as the top programming languages of 2016. The article⁵ ranked 48 programming languages (we couldn't find an appropriately matching tag for Ladder Logic in Stack Overflow) using 12 data sources.

To explore the relationship between experts in these languages, we retrieved data from the StackOverflow site using the StackExchange Data Explorer⁶ which provides a web interface for collecting data from the StackExchange network using SQL queries. We wrote queries to retrieve all questions and corresponding answers that have been tagged with the selected programming languages. We excluded closed questions and community wiki post⁷ because credit for the post cannot be awarded to any particular user. We also restricted the answers to those with at least one upvote. The dataset used was last updated on November 27, 2016.

Table 1 shows some statistics of the extracted dataset.

Description	Characteristic
Total number of questions	8, 002, 925
Total number of answers	8, 315, 842
Total number of experts	2, 749
Total number of tags	47
Date of first answer	2008-07-31
Date of last answer	2016-11-27

Table 1: Extracted Dataset Statistics

⁵<http://spectrum.ieee.org/static/interactive-the-top-programming-languages-2016>

⁶<https://data.stackexchange.com/stackoverflow/query/new>

⁷<http://meta.stackexchange.com/questions/11740/what-are-community-wiki-posts>

⁴<http://stackoverflow.com/help/whats-reputation>

3.2 Centrality Measures

3.2.1 Degree Centrality. Degree centrality is a measure of the average number of edges of a node in a graph. Figure 1 shows the graph with the nodes ranked using different sizes and colors by their degree centrality. Darker colored and larger sized nodes represent languages with higher degree.

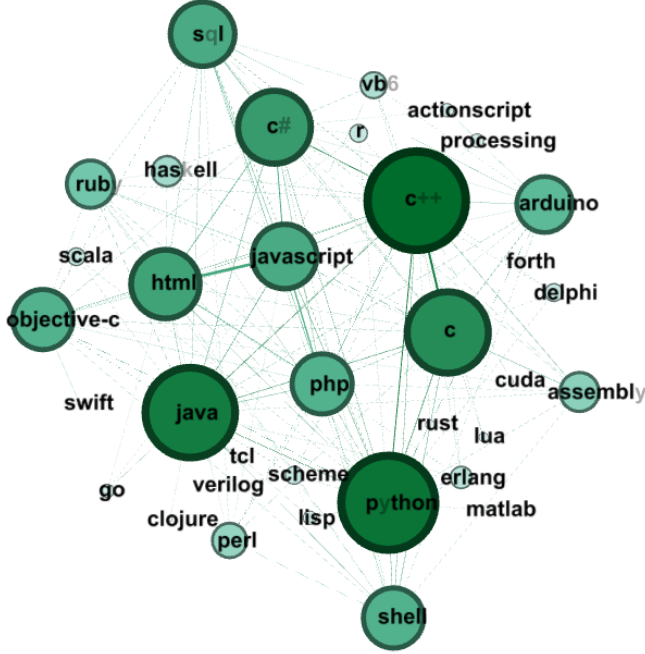


Figure 1: Programming expertise graph showing the degree centrality

3.2.2 Closeness Centrality. Closeness centrality [7] is the average shortest path between a node and all other nodes in a connected graph. Since we used weighted edges in our expertise graph, in calculating closeness centrality, we used the algorithm implemented in NetworkX [9] which replaced calculation of the path length which uses number of hops, with Dijkstra’s algorithm. Figure 2 shows the graph ranked using different sizes and colors by their closeness centrality.

3.3 Metrics for Defining Experts

An expert is someone recognized as being knowledgeable in a certain field by the public or others from the same field [19]. In this study, we defined experts for the different programming languages using two metrics: badges and weighted indegree.

3.3.1 Badges. Badges⁸ are awarded to users in Stack Overflow for being especially helpful in the community. Three types of badges (gold, silver, bronze) are awarded in seven categories: question, answer, participation, tag, moderation, other and documentation.

⁸<http://stackoverflow.com/help/badges>

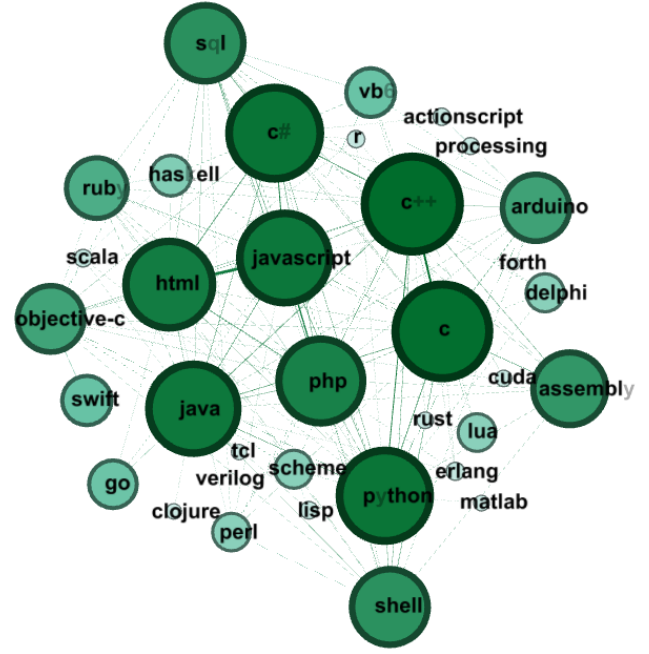


Figure 2: Programming expertise graph showing the closeness centrality

The gold badge is the most difficult to get and rewards significant contributions, while the bronze badge is the easiest to get and is used to promote new features. In this study, we define experts using the tag badges which are awarded based on the total votes a user has accumulated over time for posting non-community wiki answers. An expert in a given programming language is a user that has acquired a gold badge in that language. To obtain a gold badge, the user needs to have a total vote of 1000 in at least 200 non-community answers. In this study, there were 2,555 users with at least one gold badge.

3.3.2 Weighted In-Degree. The use of badges as a definition for expertise in different languages does not generalize well to those with smaller communities. For example, in our dataset, the language, j⁹, had only 362 answers¹⁰ with a total votes of 1,294. The top answerer for questions with this badge does not have a gold badge but have provided 22% of all answers with that tag and received 18% of the votes. Surely, this user is recognized as an expert by that programming community though not meeting the gold badge requirement. We define another metric for experts by considering a weighted directed graph for each programming language. The nodes represent users and the edges represent answer votes such that a user, X is connected to another user, Y if X has voted at least one answer provided by Y.

The weighted in-degree of a node therefore represents the recognition of skills and knowledge by users’ peers. We ranked the users

⁹<http://stackoverflow.com/tags/j/info>

¹⁰Only answers with votes greater than 0 were considered

by the percentage weighted indegree and their cumulative score. A user is an expert if they fall within the top 20% in the cumulative distribution of high scores.

3.4 Stack Overflow Expertise Graph

We wrote a python script to convert the retrieved data (Table 1) into an edge list and a node list. An edge list is a common format for representing graphs and can be imported by many graph visualization tools. It contains three columns: the first two are the source and target nodes (the programming tags in our dataset), and the third is the weight (the number of experts common to both programming languages). Each row in the file represents an edge in the graph. The node list contains a column for the nodes and another for the weight of the nodes (number of experts in that programming language). We used NetworkX [9] to compute different properties of the graph and Gephi [3] for visualization and other analyses. Table 2 describes the characteristics of the resulting graph shown in figure 3. The node size reflects the number of experts while the thickness of the edge represents the number of shared experts. Out of the 13 connected components, 12 are single nodes which represent programming languages with experts only in that language. Loopback edges (not shown in the graph) represent experts that knows only one particular language in our dataset.

Description	Characteristic
Type	Undirected
Number of nodes	47
Number of edges	144
Number of connected component	13
Size of largest component	35
Diameter	4

Table 2: Programming Language Graph Statistics

4 RESULTS

In this section, we answer the research questions defined in the introduction section and discuss the implications of our findings.

4.1 RQ1: What new programming language can be recommended to an experienced programmer?

For a novice programmer choosing a language to learn might be straightforward - for example, simply choose the language with most number of experts. As shown in Figure 3, this would mean selecting more matured languages like Java, C++, C# which have the most number of posts and thus more experts. On the other hand, a user with some experience in one or more languages and looking to learn a new one is faced with a more challenging question. What language would leverage both the user's existing knowledge and also offer a rich pool of experts to learn from?

To answer this research question, we filtered the graph (Figure 3) to retain only the largest connected components, then using Louvain method [4] for community detection we clustered the programming skills in the graph into communities as implemented in

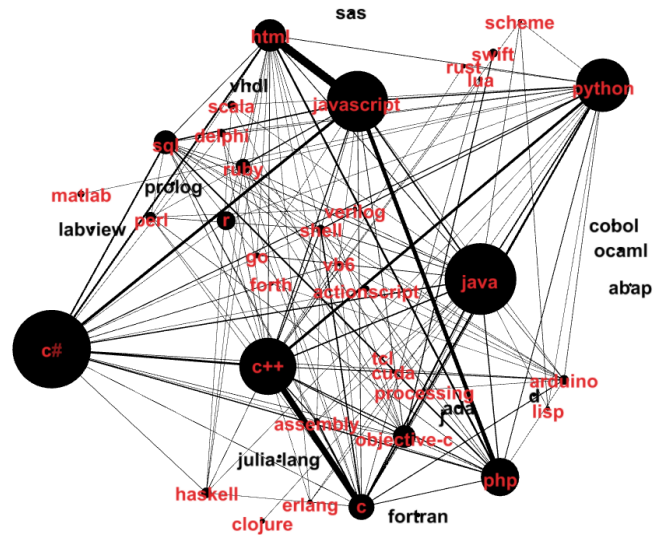


Figure 3: Programming expertise graph.

Gephi [3]. Louvain method [4] is based on optimization of modularity which measures the links density within the communities compared to that between them [15]. We used a resolution value [10] of 0.35 which resulted into 7 communities. Hence, any of these languages could be recommended to experienced programmers who have knowledge in one or more of the languages. Figure 4 shows the communities formed using different colors.

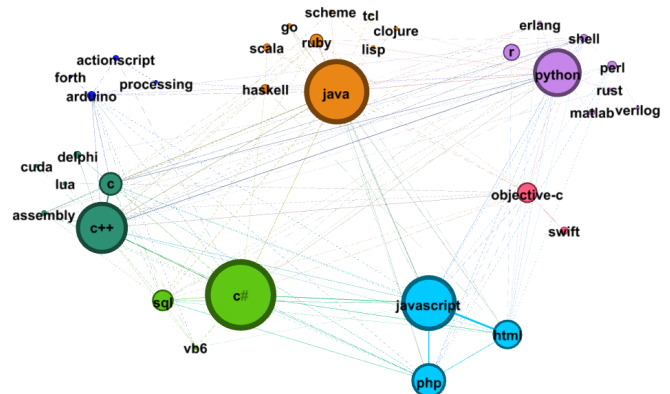


Figure 4: Programming language communities graph.

Leveraging the expertise graph to make the language recommendation allows the user benefit from a pool of experts that know the same group of programming languages which eases the transition to a new language. One implication of this finding is that authors, Massive Open Online Courses (MOOCs) developers and stakeholders can take advantage of this insight to write books; for example C++ for Java programmers [6] or develop programming courses targeting programming clusters. In addition, communities found in

the expertise graph serve as a validation that those languages can be learned together.

Other interesting findings worthy of mention include:

- Clusters are formed along corporate borders. For example, the products from Apple (Swift, Objective-C) make up a cluster while Microsoft product (C#, VB6, SQL) formed another. Although SQL¹¹ is not a product from Microsoft, a further analysis of its experts showed 45% were also experts in Microsoft SQLServer¹² and 91% of experts in SQLServer were also experts in SQL. This implication of this is that recommendations of programming languages can be made along the line of cooperate borders. For example, for a user who is experienced in C# and VB6, both Microsoft products, a recommendation of SQL could be made to such a user.
- Clusters are formed along application domains. For example in web development domain, we have languages like JavaScript, PHP and HTML. This implies that recommendations can also be made based on application domain. For example, a user who is experienced in JavaScript and HTML could recommended to learn PHP.

Table 3 lists the programming communities found in the graph. Based on communities 1 & 2, we would recommend experienced programmers stick with the same organization, for example if they are experienced in any Microsoft product, they should learn new ones also from Microsoft. Also, based on community 3, we would recommend users continue within the same development domain. For example, a web developer should learn other web development languages. No specific patterns are observed from the other communities.

Community	Programming Languages
1	Objective-C, Swift
2	C#, SQL, VB6
3	HTML, JavaScript, PHP
4	Scala, Ruby, Haskell, Java, Scheme, Clojure, Go, Lisp, Tcl
5	Matlab, R, Shell, Python, Rust, Erlang, Perl, Verilog
6	C++, Lua, C, Cuda, Delphi, Assembly,
7	Forth, Actionsript, Processing, Arduino

Table 3: Programming Language Community

4.2 RQ2: Can we infer the importance of a programming language from the expertise graph?

With a variety of languages available, can we infer from the expertise graph that some skills are more popular, supportive than others and make recommendation based on these properties? To answer

¹¹<http://stackoverflow.com/tags/sql/info>

¹²<http://stackoverflow.com/tags/sql-server/info>

this question, we applied two centrality measures: degree and closeness centrality, to the expertise graph. A high degree centrality of a node means that many other node share an edge with it, signifying that the node is very popular if we consider each edge as a popularity vote by the other nodes. In our study, if we measure importance of a programming language by popularity, our data suggests that a novice programmer learns languages such as C++, Java which both have very high degree centrality. Our data also suggests that languages with high number of experts does not always imply high popularity. For example, C programming language has a relatively low number of experts but a high degree centrality.

Our result also suggests that a programming language with high closeness provides the highest average concentration of experts from that language to all the others. Hence an expert in that language would find it relatively easier to get support to learn other languages. Our results shows that C++, C, Python and C# having the highest closeness centrality are very good first languages for novice programmers. Experts in language like Verilog and Forth would have trouble finding experts to leverage their existing knowledge.

4.3 RQ3: What can be inferred from isolated nodes and loopback edges of the expertise graph?

The graph in Figure 3 shows that 13 out of the 47 programming languages are disconnected nodes. These are programming languages whose experts have no expertise in any of the other languages in our datasets. The first thing we observed is the low popularity of these languages on the Stack Overflow site as reflected by the number of questions asked.

Further analysis of these programming tags and the profile of their experts revealed that majority of these languages are quite old (Fortran, Cobol) and although they are still highly respected today, they are not in wide spread use. An example is Ada, a very old language, whose only expert is a retired senior citizen^{13,14}. In addition, our observation suggests that for some of these languages, the experts comprise of a few devoted people committed to supporting only that language and are not interested in other languages in Stack Overflow. For example, the top two experts^{15,16} in Julia-lang are active part of the development of the language and only support Julia-lang.

In the expertise graph, many of the experts only have that level of expertise in one language. In Figure 5 we show the programming languages in the connected graph and the percentage of their experts that have expertise in other languages. The graph suggests that programmers are comfortable with being proficient in just one general purpose language like C#, Java and Python. In addition, older languages such as C and Assembly have higher percentages of experts in other languages. These languages, although powerful, do not support many features useful to developers nowadays.

¹³<http://stackoverflow.com/users/40851/simon-wright>

¹⁴<https://identi.ca/simonj>

¹⁵<http://stackoverflow.com/users/659248/stefankarpinski>

¹⁶<http://stackoverflow.com/users/3822752/iaindunning>

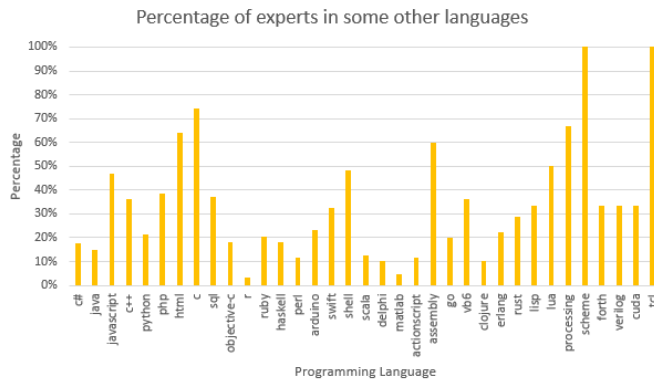


Figure 5: Percentage of experts in other languages.

4.4 Threats to Validity

In this work, we relied on observations from the expertise graph to develop our research questions; this approach may have introduced some bias in how our questions were designed. Similarly, our data was limited to a few programming languages and our inference may not be generalizable to all other languages.

5 CONCLUSION AND FUTURE WORK

With the increasing number of programming languages, knowing what language an experienced programmer should learn next can be a challenge. In this paper, we applied social network analysis techniques in recommending programming languages to experienced programmers. In particular, we created an expertise graph and used a community detection algorithm to identify clusters of programming languages which could be learned together. We then applied different centrality measures to the graph to infer importance for each of the languages. Our results suggest that programming languages can be recommended within organizational borders and programming domain. In addition, a high number of experts in a programming language does not always mean that the language is popular. Furthermore, disconnected nodes in the expertise graph includes not only older programming languages but also new ones whose experts are primarily on Stack Overflow to support that language and have no interest in contributing to unrelated questions. Finally, developers are comfortable with mastering a single general purpose language, although experts in older languages still master modern ones. In the future, we would like to include other tags used in Stack Overflow. In particular, we would include tags for all programming languages and frameworks. We believe this would help us gain more insight into the entire technology landscape which would be useful in identifying gaps in a programmer's knowledge and make recommendations to novice or experts developers. The results of our analysis can help educators, authors, developers and stakeholders of MOOCs in understanding what programming languages can be taught and learned together.

REFERENCES

[1] Alberto Bacchelli, Luca Ponzanelli, and Michele Lanza. 2012. Harnessing stack overflow for the ide. In *Proceedings of the Third International Workshop on Recommendation Systems for Software Engineering*. IEEE Press, 26–30.

[2] Anton Barua, Stephen W Thomas, and Ahmed E Hassan. 2014. What are developers talking about? an analysis of topics and trends in stack overflow. *Empirical Software Engineering* 19, 3 (2014), 619–654.

[3] Mathieu Bastian, Sebastien Heymann, Mathieu Jacomy, and others. 2009. Gephi: an open source software for exploring and manipulating networks. *ICWSM* 8 (2009), 361–362.

[4] Vincent D Blondel, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. 2008. Fast unfolding of communities in large networks. *Journal of statistical mechanics: theory and experiment* 2008, 10 (2008), P10008.

[5] Amiangshu Bosu, Christopher S Corley, Dustin Heaton, Debarshi Chatterji, Jeffrey C Carver, and Nicholas A Kraft. 2013. Building reputation in stackoverflow: an empirical investigation. In *Proceedings of the 10th Working Conference on Mining Software Repositories*. IEEE Press, 89–92.

[6] Timothy Budd. 1999. *C++ for Java Programmers*. Pearson Education India.

[7] Linton C Freeman. 1978. Centrality in social networks conceptual clarification. *Social networks* 1, 3 (1978), 215–239.

[8] Ernani Gottardo and Robinson Vida Noronha. 2012. Social networks applied to distance education courses: analysis of interaction in discussion forums. In *Proceedings of the 18th Brazilian symposium on Multimedia and the web*. ACM, 355–358.

[9] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. 2008. Exploring network structure, dynamics, and function using NetworkX. In *Proceedings of the 7th Python in Science Conference (SciPy2008)*. Pasadena, CA USA, 11–15.

[10] Renaud Lambiotte, J-C Delvenne, and Mauricio Barahona. 2008. Laplacian dynamics and multiscale modular structure in networks. *arXiv preprint arXiv:0812.1770* (2008).

[11] Laura MacLeod. 2014. Reputation on Stack Exchange: Tag, You're It!. In *Advanced Information Networking and Applications Workshops (WAINA), 2014 28th International Conference on*. IEEE, 670–674.

[12] Lena Mamkina, Bella Manoim, Manas Mittal, George Hripcsak, and Björn Hartmann. 2011. Design lessons from the fastest q&a site in the west. In *Proceedings of the SIGCHI conference on Human factors in computing systems*. ACM, 2857–2866.

[13] Patrick Morrison and Emerson Murphy-Hill. 2013. Is programming knowledge related to age? an exploration of stack overflow. In *Mining Software Repositories (MSR), 2013 10th IEEE Working Conference on*. IEEE, 69–72.

[14] Seyed Mehdi Nasehi, Jonathan Sillito, Frank Maurer, and Chris Burns. 2012. What makes a good code example?: A study of programming Q&A in StackOverflow. In *Software Maintenance (ICSM), 2012 28th IEEE International Conference on*. IEEE, 25–34.

[15] Mark EJ Newman. 2006. Modularity and community structure in networks. *Proceedings of the national academy of sciences* 103, 23 (2006), 8577–8582.

[16] Christoffer Rosen and Emad Shihab. 2016. What are mobile developers asking about? a large scale study using stack overflow. *Empirical Software Engineering* 21, 3 (2016), 1192–1223.

[17] Megan Squire. 2015. "Should We Move to Stack Overflow?" Measuring the Utility of Social Media for Developer Support. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, Vol. 2. IEEE, 219–228.

[18] Christoph Treude, Ohad Barzilay, and Margaret-Anne Storey. 2011. How do programmers ask and answer questions on the web?: Nier track. In *2011 33rd International Conference on Software Engineering (ICSE)*. IEEE, 804–807.

[19] Jie Yang, Ke Tao, Alessandro Bozzon, and Geert-Jan Houben. 2014. Sparrows and owls: Characterisation of expert behaviour in stackoverflow. In *International Conference on User Modeling, Adaptation, and Personalization*. Springer, 266–277.